An AI Based High-Precision Industrial Thermometer

Puni Aditya, Vivek K. Kumar, Jnanesh Sathisha Karmar, Burra Shreyas Goud, G.V.V. Sharma
Department of Electrical Engineering
Indian Institute of Technology Hyderabad
Kandi, India
gadepall@ee.iith.ac.in

***Abstract*—Accurate temperature measurement is critical in industrial process control. Platinum Resistance Temperature Detectors (RTDs), specifically the PT100, are widely utilized for their stability and linearity. However, traditional calibration requires expensive, specialized hardware. In this paper, we propose a cost-effective methodology for calibrating PT100 sensors using machine learning (ML) and computer vision. Our primary contribution is an Optical Character Recognition (OCR) based data acquisition pipeline that captures real-time analog thermometer readings and pairs them with sensor voltage outputs during a dynamic thermal cycle, eliminating the need for thermal chambers. Furthermore, we evaluate multiple ML techniques for non-linear calibration: physically-informed Least Squares (LSQ) regression, Stochastic Gradient Descent (SGD), and a Random Forest Regressor (RFR). Evaluated across a high-precision Wheatstone bridge with a 16-bit ADC, the physically-informed LSQ quadratic model achieves superior accuracy, yielding a Mean Squared Error (MSE) of 0.009198 and an R^2 of 0.999956, outperforming the generic regressors.**

***Index Terms*—PT100, Machine Learning, Optical Character Recognition, Temperature Measurement, Calibration, Edge Computing.**

I. Introduction

Temperature measurement is a fundamental requirement in process control, automotive systems, and instrumentation. Among the various contact temperature sensors available, the Platinum Resistance Temperature Detector (RTD), commonly known as the PT100, is considered the standard for precision thermometry [1]. PT100 sensors are widely favored in industrial applications for their linearity and long-term stability.

To achieve high accuracy, recent literature has explored both hardware and software optimizations for the PT100. Samuk and Çakır (2023) [2] implemented a high-accuracy circuit using specialized differential amplifiers. While effective, the design relies on expensive analog components and lacks an investigation into contemporary machine learning (ML) correction methods. Alternatively, Li et al. (2024) [3] focused on the design of DC-balanced Wheatstone transducers for PT100 sensors. However, their work relied entirely on software simulations, leaving real-world hardware feasibility unaddressed.

From a data-driven perspective, Liu et al. (2023) [4] successfully utilized Artificial Neural Networks (ANN) to non-linearly calibrate PT100 readings, drastically reducing error margins. Despite these advances, existing research typically demands expensive hardware testing environments or specialized dynamic testing equipment for accurate dataset generation.

To address the gap between expensive hardware calibration and software simulations, this paper presents a novel framework for PT100 calibration. First, we propose an automated Optical Character Recognition (OCR) based dataset generation pipeline that synchronizes analog readings with sensor voltages, completely circumventing the need for expensive thermal calibration chambers. Second, we present a comparative study evaluating various ML models to identify the most robust algorithm for steady-state non-linearity correction.

II. PT-100 Fundamentals

The PT100 is defined by a nominal resistance of $100\,\Omega$ at 0°C. Unlike thermocouples which generate a voltage, RTDs differ in that they operate by changing electrical resistance in direct proportion to temperature changes [5]. This relationship is inherently non-linear over wide ranges, though it is highly predictable. The resistance $R(t)$ at temperature t is governed by the Callendar-Van Dusen equation [6]

$$R(t) = R_0(1 + At + Bt^2) \tag{1}$$

for temperatures $t > 0$°C, where A and B are standard coefficients. This quadratic physical relationship serves as the theoretical justification for using LSQ models in this work.

While PT100 sensors offer high stability, the resistance change is relatively small ($\approx 0.385\,\Omega/$°C). Consequently, accurate reading requires precise signal conditioning to convert resistance changes into measurable voltage levels [7]. This paper aims to optimize this conversion by analyzing different hardware front-ends and software regression models to find an optimal solution for high-precision, low-cost temperature sensing.

III. Hardware Setup

Two main analog front-ends were tested to evaluate the trade-off between circuit complexity and measurement accuracy. Table IV lists the components required for both analog front-ends, detailing the specific parts used for high-accuracy and cost-effective implementations. The detailed hardware component list, alongside the pin-connection configurations for the ADS1115 and LCD, are provided in Appendix A to facilitate reproducibility.

- *High-Precision Wheatstone Bridge with ADC:* This uses a Wheatstone bridge configuration coupled with an external 16-bit ADS1115 ADC [8]. This setup allows for differential measurement, significantly reducing noise and common-mode errors. The connection details for integrating the external 16-bit ADS1115 ADC are available in Table V. The complete circuit schematic for the high-precision Wheatstone bridge setup is illustrated in Fig. 3.
- *Simple Voltage Divider:* This uses a simple voltage divider utilizing the Arduino's internal 10-bit ADC. The complete circuit schematic for the simple voltage divider setup is illustrated in Fig. 4.

The Arduino-LCD connections are available in Table VI. The code for the Arduino, which reads the voltage across the PT-100 and displays it on the LCD, alongside flashing instructions, is provided in our repository[1].

IV. Automated Training Using OCR

The dataset is collected by simulating a real-world thermal change (cooling cycle) to capture paired temperature readings. Since the training data is available only on an analog thermometer, instead of manually noting the temperature, we automate the data collection process using OCR based techniques. The block diagram for the same is shown in Fig. 1. Appendix B has more information on the software execution.

A. Experimental Setup

The practical requirements and experimental setup are listed below

1) *Hardware:*
   - The arduino setup with the code from the previous section flashed.
   - A phone connected to your computer.
   - (Optional) An NVIDIA GPU with CUDA installed. The script is pre-set to use `gpu=True` for much faster processing.
2) *Software:*
   - Python 3.x
   - The required Python libraries: `opencv-python`, `easyocr`, and `numpy`.

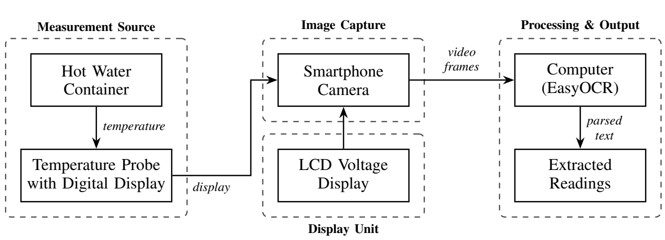

Fig. 1: Block diagram of the experimental setup

[1]Arduino Data Collection Code: https://github.com/gadepall/pt100/tree/main/codes/arduino/datacollection

B. Data Collection Methodology

1) Setup: Place both the PT100 probe and the reference thermometer inside an electric kettle, submerged in water.
2) Heating Phase: Turn on the kettle and heat the water to its boiling point.
3) Recording Phase:
   - Switch off the kettle at boiling point.
   - Start recording a video using the smartphone.
   - Ensure both the PT100 LCD display and reference thermometer are visible in the frame.
4) Cooling Phase: Continue recording the cooling process as the water temperature drops naturally.
5) Data Extraction: The recorded video acts as the raw dataset.
6) Extract frames and apply OCR to detect readings from both displays.

V. Machine Learning Models

A. Model Explanations

The LSQ and SGD models obtained from the dataset derived in the previous section are available in Table I. The coefficients are determined through linear least squares regression.

TABLE I: Extracted Polynomial Coefficients for Forward and Inverse Regression Models. (Note: X represents sensor voltage in Volts, and Y represents Temperature in °C)

Model	**Equation**	a	b	c	**Description**
Inverse LSQ (No ADC)	$X = aY^2 + bY + c$	−0.000014990	0.005589496	2.526066163	Inverse Least Squares with voltage divider setup without ADC.
Inverse LSQ (ADC)	$X = aY^2 + bY + c$	−0.000006744	0.004344843	0.056019368	Inverse Least Squares with ADC setup and Wheatstone bridge.
Quadratic LSQ (ADC)	$Y = aX^2 + bX + c$	147.376208110	197.568472526	−10.357480491	Least Squares (Quadratic) for Wheatstone bridge with ADC.
SGD	$Y = aX^2 + bX + c$	190.226107678	173.380527679	−7.072446221	Stochastic Gradient Descent model fit.
Inverse SGD	$X = aY^2 + bY + c$	−0.000005386	0.004183549	0.060452454	Inverse Stochastic Gradient Descent model fit.

1) **Least Squares (LSQ) and Inverse LSQ Regression:** Chosen as our physically-informed baseline. The Callendar-Van Dusen equation naturally dictates resistance (and analogously, voltage X) as a quadratic function of temperature (Y). We define this mathematically native orientation ($X = aY^2 + bY + c$) as the *Inverse LSQ* model. However, for real-time inference, the system must predict temperature from voltage. Thus, we also derive the *Forward LSQ* formulation ($Y = aX^2 + bX + c$). Both models use least-squares optimization to minimize the sum of squared residuals, ensuring a globally optimal fit for the sensor's physical non-linearity.
2) **Stochastic Gradient Descent (SGD) Regressor:** SGD is a common iterative optimization algorithm used to fit a linear regression model [9]. We include it to test whether a standard linear approximation can effectively map the sensor data without relying on the physical quadratic model. It also highlights the necessity of proper data normalization in ML pipelines.
3) **Random Forest Regressor (RFR):** RFR is a standard ensemble learning method based on decision trees [10]. It

is included to evaluate if a generalized non-linear machine learning model can outperform the physically based LSQ approach. Because tree-based models rely on discrete spatial partitioning (step functions), we investigate their ability to smoothly interpolate continuous analog variables like temperature.

The models are obtained and implemented using Python and the scikit-learn [11] library. The implementation of these models and the evaluation of Mean Squared Error (MSE) and R-squared (R^2) metrics are available in the repository[2].

VI. Model Evaluation and Comparison

Table II summarizes the Mean Squared Error (MSE) and R-squared (R^2) scores for all evaluated models [12]. MSE was specifically selected to heavily penalize larger temperature prediction deviations—a critical safety requirement in industrial control systems. Conversely, the R^2 metric is utilized to measure the proportion of variance in the actual temperature that is predictably explained by the sensor's input voltage. A robust industrial thermometer must achieve an MSE approaching zero and an R^2 approaching 1.0. Based on these metrics, the LSQ (Quadratic) model for the Wheatstone bridge with ADC vastly outperforms the rest.

TABLE II: Low MSE and high R^2 is desirable

Model	MSE	R^2
1. Inverse LSQ (Volt. Div w/o ADC)	0.6612	0.9977
2. Inverse LSQ (Wheatstone w/ ADC)	0.016221	0.999922
3. LSQ (Wheatstone w/ ADC)	0.009198	0.999956
4. SGD Regressor (Wheatstone w/ ADC)	0.030477	0.999854
5. Inverse SGD (Wheatstone w/ ADC)	0.016221	0.999922
6. Random Forest (Wheatstone w/ ADC)	1.161318	0.994451

Table III presents a small subset of the test data, showing the actual temperature values alongside the predictions from the Random Forest, Inverse LSQ, LSQ, SGD, and Inverse SGD Regressor models for the Wheatstone bridge setup with ADC. The corresponding graph is available in Fig. 2. We thus conclude that the LSQ model is best at predicting the temperature. Though the results are provided only for a small range of temperatures in Table III and Fig. 2 for brevity and visualization respectively, the results hold true for all values of practical interest.

TABLE III: Wheatstone Validation Data and Predictions

Volt (V)	Actual (°C)	Inv LSQ (ADC)	LSQ (ADC)	SGD (ADC)	Inv SGD (ADC)	RF (ADC)
0.2634	51.9	51.9136	51.9070	51.7938	51.9906	53.3456
0.3127	65.9	65.7972	65.8329	65.7442	65.8829	67.6248
0.2083	37.2	37.1962	37.1906	37.2964	37.1134	37.0563
0.4037	93.1	93.6290	93.4194	93.9231	93.2383	89.0505
0.2092	37.5	37.4305	37.4238	37.5239	37.3513	37.0563
0.2978	61.6	61.5231	61.5485	61.4304	61.6219	61.8901
0.2028	35.8	35.7687	35.7707	35.9127	35.6628	36.7501
0.2666	52.7	52.7931	52.7892	52.6712	52.8748	53.3456
0.3185	67.5	67.4803	67.5183	67.4462	67.5568	68.7954
0.2850	57.9	57.9067	57.9202	57.7921	58.0053	58.6497

[2]Model Implementation: https://github.com/gadepall/pt100/blob/main/codes/models/PT100_models.ipynb

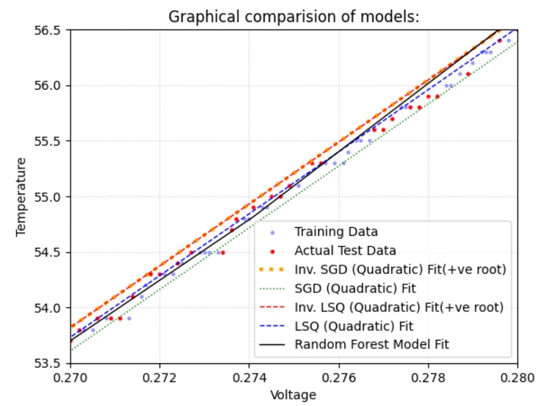

Fig. 2: Comparison of Random Forest, Inverse LSQ, LSQ, SGD and Inverse SGD Regressor predictions against actual test data for the Wheatstone bridge with ADC for temperatures between 53.5°C to 56.5°C

The code for the Arduino, which predicts the temperature from the voltage and flashing instructions are available in the repository[3].

Appendix A
Hardware Configuration and Schematics

TABLE IV: Component List

Component	Qty.	Description
Arduino Uno	1	Microcontroller for processing and control.
ADS1115 ADC	1	16-bit external ADC for high-precision voltage measurement.
PT100 RTD Sensor	1	Platinum resistance thermometer.
100Ω Resistor	1	Precision resistor for Wheatstone bridge.
4kΩ Resistors	2	Resistors for Wheatstone bridge.
JHD 162A Parallel LCD	1	For displaying the final temperature.
22kΩ Potentiometer	1	To adjust the contrast of the LCD.
Breadboard	1	For creating solderless circuit connections.
Jumper Wires	Set	For connecting all the components.
Pen Thermometer <= 0.1°C resolution	1	Used for training purpose and as a reference.
Kettle/Heater with water	1	To heat the water, so that training data corresponding to a wide range of temperatures can be obtained

TABLE V: ADS1115 Connections

ADS1115 Pin	Arduino Pin	Purpose
VDD	5V	Power Supply
GND	GND	Common Ground
SCL	A5 (SCL)	I2C Clock Line
SDA	A4 (SDA)	I2C Data Line
A0	Input from Voltage Divider Node (between PT100 and R_REF)	Reading Voltage
A1	Junction between the two 4kΩ resistors	Reference for differential reading

[3]Temperature Prediction: https://github.com/gadepall/pt100/codes/arduino/inference

TABLE VI: LCD Connections

LCD Pin	Arduino Pin
VSS	GND
VDD	5V
V0	Potentiometer Middle Pin
RS	Digital 12
RW	GND
E	Digital 11
D4	Digital 5
D5	Digital 4
D6	Digital 3
D7	Digital 2
A (Anode)	5V
K (Cathode)	GND

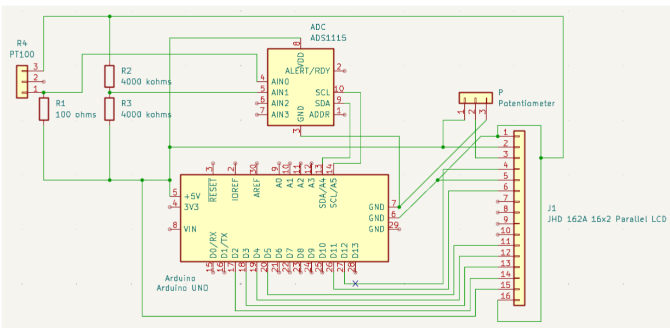

Fig. 3: Circuit Schematic of the High-Precision Wheatstone Bridge

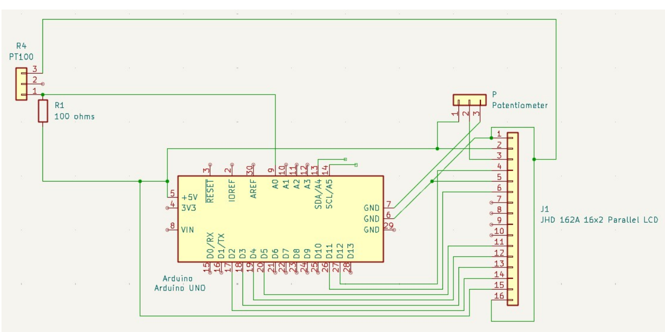

Fig. 4: Circuit Schematic of the simple voltage divider

Appendix B
Automated OCR Training Pipeline Setup

A. *Pipeline Architecture and Data Logging*

The automated data acquisition pipeline was developed using Python. To capture the real-world thermal cycle, a smartphone camera was interfaced with a computer via DroidCam and OBS Studio, providing a continuous video feed of both the analog reference thermometer and the sensor's LCD display. The text extraction was handled using the `easyocr` [13] library, which relies on PyTorch [14] for deep learning-based inference, accelerated via an NVIDIA GPU.

During the cooling phase of the thermal cycle, the script continuously monitored the video feed. Every 15 frames, if text was successfully detected in both regions, the script synchronized the readings and logged them into a structured dataset, formatted as paired voltage and temperature values. To facilitate manual debugging, the isolated bounding boxes of the digits were saved as individual image frames. Because the OCR engine is susceptible to occasional character misclassification, the generated dataset (`out.txt`) was manually reviewed and corrected to produce the final high-fidelity training dataset.

B. *Image Processing and Region of Interest (ROI) Calibration*

To achieve high OCR accuracy, the raw video frames required specific preprocessing and spatial cropping. The camera feed was divided into distinct Regions of Interest (ROIs): the voltage display was isolated from the top half of the frame, while the physical thermometer was isolated from the bottom-left third.

Because the analog thermometer display was highly sensitive to ambient lighting, standard grayscaling was insufficient for reliable digit extraction. Therefore, an inverse binary thresholding operation was applied (with a threshold value of 130) to separate the liquid crystal digits from the background. To reconstruct broken segments within the digital numbers, a morphological closing operation was executed utilizing an 11×11 rectangular structuring element. Finally, a 17×17 Gaussian blur was applied to the inverted mask to smooth the edges of the characters, significantly reducing background noise before the frames were passed to the `easyocr` engine. The further information regarding the installation and setup is provided in https://github.com/gadepall/pt100/tree/main/AutoTrainer .

References

[1] T. Bartelt, *Industrial Control Electronics: Devices, Systems & Applications*. Delmar Cengage Learning, 2005.

[2] A. Samuk and M. Çakır, "New circuit design and implementation for temperature measurement based on pt100 sensor," *ResearchGate*, 2023.

[3] M. Li, C. Zeng, and Y. Wang, "Pt100 temperature measurement sensors design and transducer simulation," *Journal of Physics: Conference Series*, vol. 2897, no. 1, p. 012011, 2024.

[4] C. Liu, C. Zhao, Y. Wang, and H. Wang, "Machine-learning-based calibration of temperature sensors," *Sensors*, vol. 23, no. 17, p. 7347, 2023.

[5] J. Fraden, *Handbook of Modern Sensors: Physics, Designs, and Applications*. Springer, 2016.

[6] H. L. Callendar, "On the practical measurement of temperature," *Philosophical Transactions of the Royal Society of London. A*, vol. 178, pp. 161–230, 1887.

[7] T. Mien and N. Hien, "Precision temperature measurement using wheatstone bridge," in *2019 International Conference on System Science and Engineering (ICSSE)*. IEEE, 2019, pp. 145–149.

[8] *ADS111x Ultra-Small, Low-Power, I2C-Compatible, 860-SPS, 16-Bit ADCs*, Texas Instruments, 2018, sBAS444D.

[9] H. Robbins and S. Monro, "A Stochastic Approximation Method," *The Annals of Mathematical Statistics*, vol. 22, pp. 400 – 407, 1951.

[10] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[11] F. e. a. Pedregosa, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[12] Y. M. e. a. Jaradat, "Beyond one-size-fits-all: Comparing and selecting regression metrics for robust model assessment," in *2025 12th International Conference on Information Technology (ICIT)*, 2025, pp. 416–422.

[13] JaidedAI, "Easyocr: Ready-to-use ocr with 80+ supported languages," 2020.

[14] A. e. a. Paszke, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 8024–8035.